QUANT_CHART

Native PyQt6 Footprint Charting Platform

Project & code-structure reference · generated for the team

This document explains how the application is organised, how the pieces fit together, and — file by file — what each module exposes and what it hands back to the code that uses it. It is meant as the shared map: read it before touching the project or before another chat session picks up the work.

1 · What the application is

A desktop research tool for reading order flow on intraday futures. It renders enriched 1-minute candles as an interactive **footprint chart** drawn with QPainter: per-price buy/sell volume, resting (passive) liquidity, volume delta, anchored VWAPs, cumulative delta, large-trade bubbles, and several kinds of volume profile. On top of the chart sit manual drawing tools (lines, rays, boxes) and TradingView-style long/short position tools.

It is a single desktop process. There is no server and no web layer — just Python, PyQt6, pandas/numpy for data, and Parquet files on disk as the data source.

How it runs

```
python main.py
```

```
main.py  →  QApplication + dark theme  →  MenuWindow (home)
                                                  │
          pick data + open a chart  ──────────────┘
                                                  ↓
                                          FootprintWindow
```

There is **no hot reload**. After editing any file, fully restart `python main.py`.

Tech stack

• **Python + PyQt6** — UI and the QPainter chart canvas. • **pandas / numpy** — data loading and in-memory models. • **Parquet** — on-disk candle / indicator / big-trade files. • **QSettings** — persists window/menu state and all visual settings (no config file to hand-edit).

2 · Folder structure

One organising rule decides where a file lives:

core/ = shared by *any* chart or feature. **views/<feature>/** = used only by that one feature.

```
Quant_chart/
├─ main.py                        app entry point
│
├─ core/                          ◂ shared, chart-agnostic
│  ├─ asset_info.py               tick sizes per instrument
│  ├─ data_paths.py               folder / asset / dataset / date listing
│  ├─ settings.py                 menu (home-screen) state
│  ├─ styles.py                   app-wide dark theme
│  └─ viewport.py                 coordinate system + pan/zoom (class Viewport)
│
└─ views/
   ├─ menu/
   │  └─ menu.py                  home window, routes to charts
   └─ footprint_chart/            ◂ everything specific to the footprint chart
      ├─ footprint_config.py       the resolved selection (class FootprintConfig)
      ├─ footprint_data.py         candle / indicator / big-trade loaders + models
      ├─ open_dialog.py            the 'Open chart' pop-up
      ├─ volume_profile.py         peak-based volume-profile algorithm
      ├─ footprint_settings.py     persisted visual settings (dataclasses)
      ├─ settings_dialog.py        the settings window
      ├─ canvas.py                 the QPainter chart widget
      └─ window.py                 toolbar + canvas container
```

Future chart types slot in as siblings of `footprint_chart/` (e.g. `views/ohlcv_chart/` with its own `ohlcv_config.py` / `ohlcv_data.py`), reusing everything in `core/`.

3 · How the pieces talk to each other

The flow from a click on the home screen to a rendered chart:

```
MenuWindow (menu.py)
   user picks root / asset / dataset / date / time filter
           │  opens
           ▼
OpenChartDialog (open_dialog.py)
   confirms selection, optional indicators + big-trades datasets
           │  returns
           ▼
FootprintConfig (footprint_config.py)          ◂— a plain dataclass: the 'what to load'
           │  passed to
           ▼
FootprintWindow (window.py)
   load_candles(config)        → ChartData
   load_indicators(config, …)  → IndicatorData
   load_big_trades(config, …)  → BigTradeData
           │  set_data / set_indicators / set_big_trades
           ▼
FootprintCanvas (canvas.py)
   owns a Viewport (core/viewport.py) for all pixel↔price↔index math
   reads settings dataclasses (footprint_settings.py)
   calls compute_profile / load_composite_volume on demand
   → paints everything
```

Key idea: **data flows one way**. Loaders turn a config into immutable in-memory models; the canvas only reads those models plus the settings objects and the viewport. Nothing downstream writes back to the Parquet files.

4 · File-by-file reference

For each file: its job, and the public surface other modules rely on — including **what it returns**.

core/ — shared

`core/asset_info.py`
Static lookup of contract tick sizes. The single source of truth for price granularity.

Exposes	returns
`tick_size_for(asset, default=0.25)`	float — the tick size for a ticker (e.g. ES → 0.25). Falls back to `default` for unknown assets.
`TICK_SIZES`	dict[str, float] — the raw table, keyed by uppercase ticker.

`core/data_paths.py`
Filesystem discovery for the cascading selectors. All functions are fail-soft: a missing folder returns an empty list rather than raising.

Exposes	returns
`list_type_folders(root)`	list[str] — top-level type folders (Futures, …).
`list_assets(root, type_folder)`	list[str] — asset tickers under a type.
`list_datasets(root, type_folder, asset)`	list[str] — dataset folders for an asset.
`list_dates(root, type_folder, asset, dataset)`	list[str] — available `YYYY-MM-DD` file dates.

`core/settings.py`
Thin wrapper over QSettings for the home screen's remembered state (data root, asset, date, time filters). Survives restarts. Visual chart settings live elsewhere (footprint_settings.py).

Exposes	returns
`AppSettings().get(key, default, type_)`	the stored value, typed.
`AppSettings().set(key, value)`	None — writes & syncs immediately.
`Keys`	central constants for the setting keys (ROOT, ASSET, DATE, TIME_START, …).

`core/viewport.py`

The coordinate system shared by every chart. Holds the visible price window plus pan/zoom state, and converts between data space (candle index, price) and screen space (pixels). Renderers mutate and read this one object so the whole chart stays consistent. (Renamed from view_transform.py.)

Exposes	exposes
`class Viewport`	state: offset_x, candle_w, center_price, scale_y, price_min/max, and the screen rect left/top/right/bottom (set by the widget each paint).
`price_to_y(price) / y_to_price(y)`	float — vertical pixel↔price mapping.
`candle_x(i) / x_to_candle_round(x) / x_to_candle_floor(x)`	horizontal index↔pixel mapping.
`tick_px(tick) / pixels_per_price() / price_span()`	float — scale helpers used for sizing.
`GAP_RATIO`	class constant — gap between candle slots as a fraction of candle width (0.025).

views/footprint_chart/ — the chart

`views/footprint_chart/footprint_config.py`
A plain dataclass describing what to load: the resolved menu selection handed to a chart window. Carries no behaviour beyond convenient path helpers.

Exposes	exposes
`class FootprintConfig`	fields: root, type_folder, asset, dataset, date, days_back, time_start, time_end, tf_value, tf_unit, indicators_dataset, big_trades_dataset.
`dataset_path() / indicators_path() / big_trades_path()`	str (or None) — resolved folders on disk.
`has_dataset() / has_indicators() / has_big_trades()`	bool — whether that layer is selected.
`timeframe_str()`	str — e.g. '5 Minutes', for display.

`views/footprint_chart/footprint_data.py`
All disk → memory loading for the footprint chart, plus the in-memory models. Reads daily Parquet files, stitches the requested day-span in chronological order, applies the time-of-day filter and (if needed) resamples to the chart timeframe. This is the file that defines the enriched candle schema.

Exposes	returns
`load_candles(config)`	ChartData | None — the candle model for the selection.
`load_indicators(config, candle_times)`	IndicatorData | None — indicators resampled and aligned 1:1 to the candle bars.
`load_big_trades(config, days_back)`	BigTradeData | None — individual large trades, time-sorted.
`load_composite_volume(config, days_back, end_time)`	(levels: dict{price→total volume}, days_loaded: int) — multi-day total-volume profile; the last day is cut at `end_time` (exclusive).
`select_day_files(folder, date, days_back)`	list[Path] — the day-files to load, chronological.
`resample_dataframe(df, tf_value, tf_unit)`	DataFrame — 1-minute candles aggregated to a timeframe.

In-memory models produced above (what the canvas actually reads):

Exposes	exposes
`ChartData`	numpy arrays o/h/l/c, volume, buy/sell volume, delta, delta_pct; lists tick_volume & passive_orders (per-price dicts); times (tz-aware) & times_ns; session_starts; n / len().
`IndicatorData`	.has(col) → bool, .col(col) → numpy float array aligned to bars (VWAP bands, cumulative_delta, absorption_score, …).
`BigTradeData`	time-sorted arrays price, size, side, ts_ns, tod_min (minutes-of-day NY); n / len(). RTH classification is done at draw time so the configurable window applies without reloading.

views/footprint_chart/volume_profile.py
The peak-based volume-profile algorithm, shared by the manual / ETH / RTH profiles on the chart. Smooths the per-price volume, finds clustered peaks, expands each to a 70% value area and keeps the tightest — giving meaningful POC/VAH/VAL even on bimodal profiles.

Exposes	returns
`compute_profile(data, start_idx, end_idx, tick_size)`	Profile | None — a profile over a candle range.
`class Profile`	fields: start_idx, end_idx, levels (price→{total,buy,sell}), poc, vah, val, max_total, total.

views/footprint_chart/footprint_settings.py
All persisted *visual* settings as dataclasses, each with .load() (from QSettings) and .save(). These are read live by the canvas, so changing one and repainting updates the chart without reloading data.

Exposes	exposes
`OrderFootprintSettings`	orders footprint imbalance gradient + highlight thresholds.
`VolumeFootprintSettings`	volume footprint gradient + show-delta toggle.
`PassiveOrderSettings`	resting-liquidity gradient + highlight.
`BigTradeSettings`	bubble sizing, ETH/RTH min-contract filters, days_back.
`GeneralSettings`	RTH session start (incl) / end (excl) — shared by big trades & ETH/RTH profiles.
`VolumeProfileSettings`	RTH profile minutes, composite days/end, and the vol/delta/composite bar widths.

`views/footprint_chart/open_dialog.py`
The ‘Open chart’ pop-up. Lets the user confirm the dataset and pick optional indicator and big-trade datasets before the chart opens.

Exposes	exposes
`class OpenChartDialog`	a QDialog; on accept it builds and stores a FootprintConfig.
`.config`	FootprintConfig — the resolved selection read back by the menu after the dialog closes.

`views/footprint_chart/settings_dialog.py`
The categorised settings window (General, Footprint, Volume Footprint, Passive, Volume Profile, Big Trades). Edits the settings dataclasses live, persists them, and fires callbacks so the chart repaints or reloads (e.g. composite reload when its days/end-time change).

Exposes	exposes
`class FootprintSettingsDialog`	non-modal QDialog; takes the settings objects + on_change / on_bt_reload / on_composite_reload callbacks.

`views/footprint_chart/canvas.py`
The heart of the app: the QPainter widget that draws candles and every overlay, and handles all mouse interaction (pan, zoom, drawing tools, editing, deletion, tooltips). Owns a Viewport and reads the data models + settings.

Exposes	exposes (called by window.py)
`set_data(data, tick_size, focus_last_session=True)`	load a ChartData and fit the view.
`set_indicators(indicators) / set_big_trades(big_trades)`	attach the optional layers.
`toggle_layer(name)`	show/hide footprint, volume, passive, vwap, cvd, big_trades.
`set_vwap_type(vtype)`	choose which anchored VWAP to draw.
`add_session_profiles(kind)`	add ETH or RTH volume profiles, one per loaded session.
`set_composite(levels, days) / clear_composite()`	show/hide the right-side composite profile.
`set_draw_mode(mode)`	arm a tool: hline, ray, vline, box, long, short, edit, delete, vp.
`clear_all()`	remove all drawings (lines, rays, boxes, positions, profiles).

`views/footprint_chart/window.py`
The chart window: builds the toolbar (layer toggles, VWAP selector, Volume-Profile dropdown, drawing-tool icons, settings gear), loads the data via the loaders, and wires toolbar actions to the canvas.

Exposes	exposes
`class FootprintWindow`	QMainWindow; constructed with a FootprintConfig, opens maximised, and drives the canvas.

views/menu/ & entry point

`views/menu/menu.py`
The home window. Cascading selectors for data root / type / asset / dataset / date plus time filters, and buttons to open a chart. Launches OpenChartDialog, then a FootprintWindow from the returned config.

Exposes	exposes
`class MenuWindow`	the home QMainWindow — the first thing main.py shows.

`main.py`
Entry point. Creates the QApplication, applies the dark stylesheet, shows the MenuWindow, runs the event loop.

5 · Data on disk

The chart never writes to these files. One Parquet file per trading day, indexed by a tz-aware America/New_York timestamp, covering the full Globex session (18:00 previous evening → 17:00).

```
data/parquet/{type}/{asset}/{dataset}/YYYY-MM-DD.parquet
   e.g.  .../Futures/ES/ES_1m_advanced/2025-04-30.parquet
```

Dataset folder	What it holds
`ES_1m_advanced`	Enriched 1-minute candles: OHLC, volume, buy/sell volume, delta, delta_pct, tick_volume (per-price buy/sell JSON), passive_orders (resting liquidity JSON).
`ES_1m_indicators`	Same daily filenames: 28 VWAP columns (4 anchors × bands), cumulative_delta, and absorption-score columns (beta, residual, absorption_score).
`ES_big_trades`	One row per large trade: price, size, side (B/A/N), tz-aware timestamp index.

6 · Conventions & gotchas

• **core vs view:** if a second chart type would need it, it belongs in `core/`; otherwise it lives under that chart's folder.

• **One-way data flow:** loaders return immutable models; the canvas only reads. Never mutate loaded data in place.

• **Settings are live:** visual settings are read on every paint, so a change + repaint is enough — except composite days/end-time and big-trade days_back, which reload data via callbacks.

• **RTH window is shared:** GeneralSettings.rth_start/rth_end drive both the big-trade ETH/RTH split and the ETH/RTH volume profiles. Start is inclusive, end exclusive.

• **Timestamps:** everything is tz-aware America/New_York; `times_ns` (epoch-ns) is used to map trades to candles — resolution-independent, so don't reintroduce `.view('int64')` hacks.

• **No hot reload:** restart `python main.py` after edits.

• **Direction strings** for positions/strategies are lowercase ('long' / 'short'); the chart and downstream tools match exactly.